AeroNet Lite: Autonomous Drone Delivery Simulation

Artificial Intelligence

Semester Project Report

Submission Date: May 10, 2026

Project Team

Amaim Anwar	23i-2614
Dania Waseem	23i-2622
Munaza Tariq	23i-2545
Taiba Tariq	23i-2618

Department of Data Science

National University of Computer and Emerging Sciences
Islamabad, Pakistan

,

Contents

List of Figures

List of Tables

Contents

0.1 Introduction

AeroNet Lite is a fully integrated autonomous drone delivery simulation developed as the semester project for the Artificial Intelligence course (BSDS, SP2026). The system models a 10×10 urban city grid and simulates the complete lifecycle of drone deliveries, from city layout validation through optimised route planning, a 20-step live simulation, machine learning driven demand forecasting, anomaly detection, and an interactive Streamlit dashboard.

The project is divided into six modules, each built on top of the previous:

- **Module 1 — Shared Grid Model:** Defines the city grid, zone types, hubs, charging pads, and all visualisation utilities.
- **Module 2 — CSP Layout Validator & Fleet Selector:** Enforces four hard spatial constraints on the grid and selects an optimal drone fleet under a budget using Brute-Force and Genetic Algorithm search.
- **Module 3A — A* Path Planner:** Finds least-cost delivery routes on the zone-weighted grid, handling no-fly zones and mid-flight disruptions.
- **Module 3B — Delivery Simulator:** Runs the 20-step tick-based simulation with drone movement, disruption handling, battery monitoring, and anomaly response.
- **Module 4 — ML Pipeline:** Trains a Random Forest demand-forecasting regressor and a multi-class anomaly classifier, integrating their outputs into the live simulation.
- **Module 6 — Streamlit Dashboard:** Provides an interactive web interface tying all modules together for real-time control and visualisation.

The project demonstrates how AI techniques, including constraint satisfaction, heuristic search, evolutionary algorithms, and supervised machine learning, combine to address a realistic urban logistics problem.

0.2 Module 1 — Shared Grid Model

0.2.1 Overview

The Shared Grid Model forms the foundation of the AeroNet Lite simulation. It defines the 10×10 city grid that every downstream module depends on. No fleet selection, route planning, or simulation can proceed until the grid is built and validated. The

grid is implemented in `grid_model.py` and the sample city configuration is stored in `data/raw/sample_grid.json`.

0.2.2 Cell Properties

The grid is a 10×10 matrix where each cell is a Python dataclass with the following properties:

Table 1: Cell attributes in the Grid Model

Property	Type	Purpose
`row, col`	int	Cell position (0 to 9 for each axis)
`zone`	string	Zone type: residential, commercial, hospital, school, industrial, open_field
`density`	int	Population density per zone (0 to 10,000)
`demand`	float	Normalised delivery demand (set from density at grid creation)
`is_hub`	bool	Marks drone dispatch hub locations
`is_charging`	bool	Marks charging pad locations
`is_medical_pickup`	bool	Marks medical supply pickup points
`no_fly`	bool	Blocks A* routing through this cell

0.2.3 Zone Types and Demand Weights

Each zone is assigned a population density which is used to derive its normalised demand value. Residential zones carry the highest demand since they represent the largest number of end recipients.

Table 2: Zone types, densities, and normalised demand values

Zone	Density	Demand (normalised)	Grid colour
Residential	5,000	1.00	Light blue
Commercial	3,000	0.60	Orange
Hospital	2,000	0.40	Red
School	1,500	0.30	Green
Industrial	1,000	0.20	Grey
Open Field	0	0.00	White

0.2.4 Grid Methods

The `Grid` class provides the following core methods used by all other modules:

- `get_cell(row, col)` — Returns the cell at the given position, or `None` if out of bounds.
- `get_neighbors(row, col)` — Returns valid 4-directional neighbours, handling edge and corner cells correctly.
- `get_cells_by_type(...)` — Filters cells by zone, hub, charging, or other properties.
- `get_hub_positions()` — Returns a list of all hub cell coordinates.
- `manhattan_distance(pos1, pos2)` — Returns $|r_1 - r_2| + |c_1 - c_2|$.
- `set_cell(...)` — Updates cell properties (used to activate no-fly zones during simulation).

0.2.5 City Grid Layout

The sample grid contains six drone hubs placed to ensure that every residential cell lies within Manhattan distance 3 of at least one hub (satisfying CSP rule R2). Six charging pads are placed within distance 2 of each hub (R3). A hospital at cell (2,4) carries a medical pickup flag (R4). Industrial zones at rows 3 to 5, columns 6 to 7 are isolated from schools and hospitals (R1).

Figure 1: AeroNet Lite city grid, zone map with hub markers (H), charging pads (+), medical pickup (M), and zone abbreviations in each cell.

0.2.6 Visualisation

All figures are generated by `visualization.py` using Matplotlib and saved automatically to `results/figures/`. Four plot types are provided:

- **Grid Map** (`plot_grid`) : Colour-coded zone map with all facility markers and zone name labels.
- **Route Overlay** (`plot_route`) : Draws a coloured arrow path on the grid for a given drone route.
- **Demand Heatmap** (`plot_heatmap`) : YlOrRd colour scale showing demand intensity per cell.
- **Simulation Dashboard** (`plot_all`) : Four-panel figure combining the zone map, drone routes, demand heatmap, and delivery statistics.

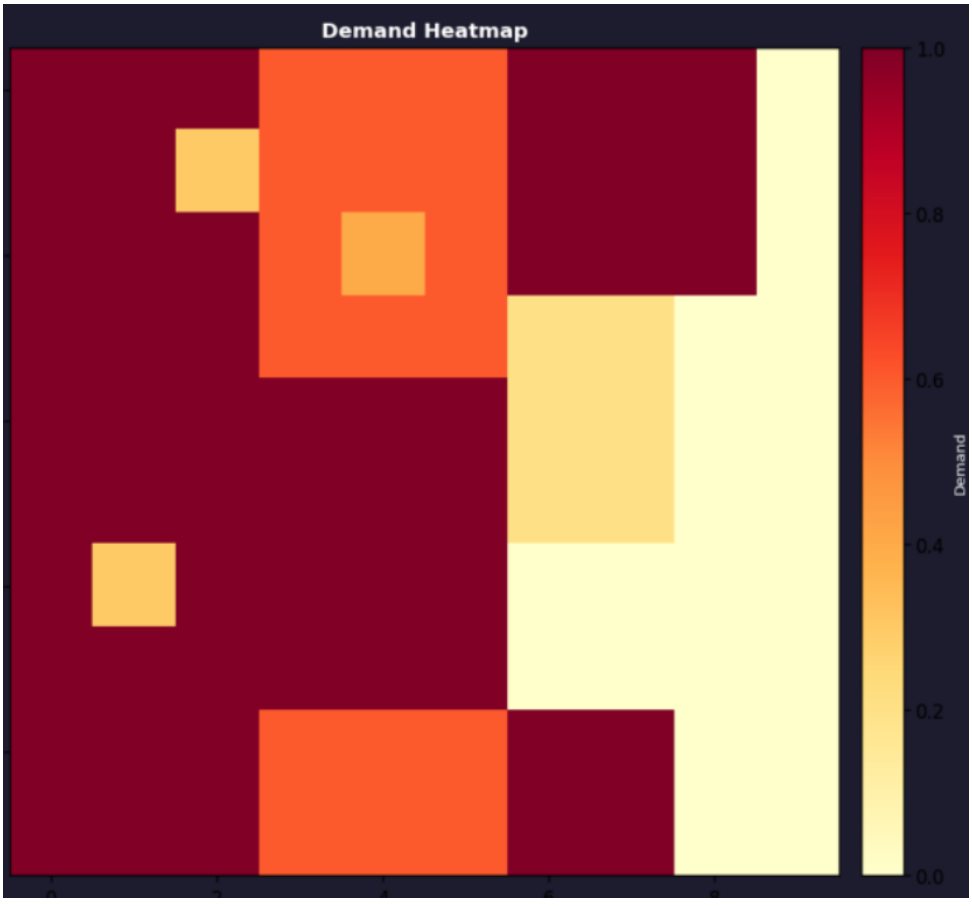


Figure 2: Demand heatmap : darker red cells indicate higher delivery demand. Residential zones (demand = 1.0) are darkest; open field cells (demand = 0) are white.

0.2.7 Integration with the Simulation

The grid model is the first thing initialised when `main.py` runs. All five other modules import from `grid_model.py` directly ; the validator reads zone and hub data, the fleet selector reads total demand, the A* planner reads zone costs and no-fly flags, and the simulator reads hub positions for emergency drone returns.

0.3 Module 2A — CSP Layout Validator

0.3.1 Overview

The Layout Validator ensures that the city grid conforms to all operational constraints before any simulation can proceed. This is implemented as a Constraint Satisfaction Problem (CSP), an AI technique where a solution must simultaneously satisfy a set of hard constraints. The validator checks all four rules independently, collects every violation with exact cell coordinates and suggested fixes, and blocks the pipeline if any rule fails.

0.3.2 The Four Constraint Rules

Table 3: CSP constraint rules in `layout_validator.py`

Rule	Name	Constraint	How Checked
R1	Industrial Safety	Industrial cells must NOT be directly adjacent to Schools or Hospitals	Check 4 direct neighbours of each industrial cell
R2	Residential Coverage	Every residential cell must be within Manhattan distance 3 of a drone hub	Compute distance from each residential cell to all hubs
R3	Hub Charging	Every drone hub must have a charging pad within Manhattan distance 2	Compute distance from each hub to all charging pads
R4	Medical Access	At least one hospital must have a medical pickup within distance 1	Check all hospital cells against all medical pickup cells

0.3.3 Helper Functions

Two helper functions are shared across all four constraint checks:

- `get_neighbors(row, col)` : Returns the list of valid 4-directional neighbours for any cell, correctly handling edge and corner cells that have fewer than four neighbours.

- `manhattan(a, b)` : Returns $|r_1 - r_2| + |c_1 - c_2|$, the Manhattan distance between two cells. This is the standard admissible distance metric for 4-directional grid movement.

0.3.4 Validation Report

The `validate_layout()` function returns a dictionary containing: `valid` (True only if all four rules pass), `passed_rules`, `failed_rules`, `errors` (violation messages per rule with suggested fixes), and `summary` (single sentence result). The full report is printed to the terminal and saved to `results/validation_report.txt`.

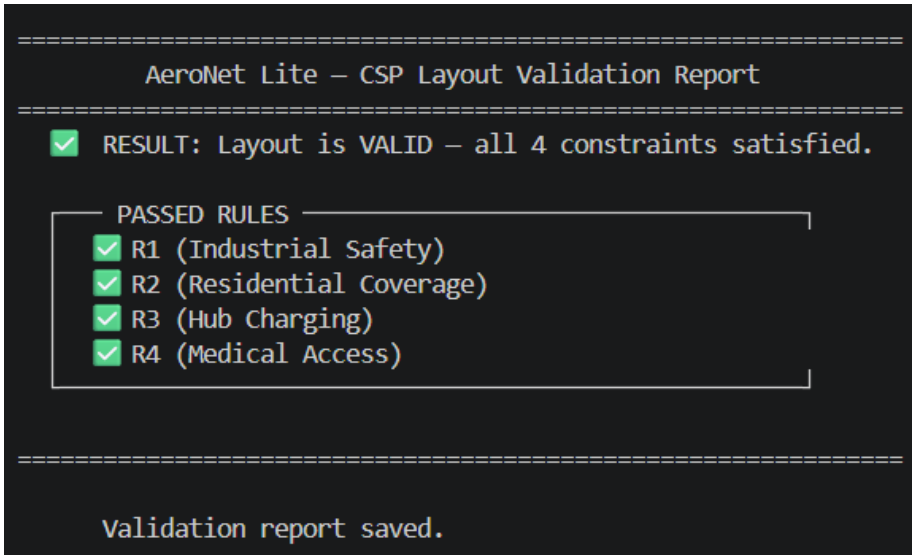


Figure 3: CSP validation report, all four rules passing. The validator reports each rule individually and stops the pipeline if any rule fails.

0.3.5 Integration with the Simulation

The validator executes in the first two steps of the 20-step simulation:

Table 4: Validator integration in the simulation

Step	Module	What Happens
Step 1	Layout Validator	All 4 CSP constraints are checked. Passed and failed rules are reported with cell-level details.
Step 2	Layout Validator	If all rules pass, proceeds to fleet selection. If any rule fails, simulation stops.
Step 3+	Other Modules	Fleet selection, route planning, and simulation use the validated grid.

0.4 Module 2B — Fleet Selector

0.4.1 Overview

The Fleet Selector finds the most effective mix of Light and Heavy drones within a given budget. It identifies a fleet that maximises delivery coverage across the grid while staying under financial constraints. Two strategies are implemented: Brute-Force search (always optimal) and a Genetic Algorithm (scalable for larger search spaces).

0.4.2 Drone Types

Table 5: Drone specifications

Type	Cost	Payload	Range	Use Case
Light Drone	1,000	2 kg	12 cells	Small parcels, short-range
Heavy Drone	1,800	5 kg	20 cells	Medical supplies, long-range

0.4.3 Fitness Function

Every candidate fleet is scored using:

$$\text{score} = 0.75 \times \text{coverage\%} - 0.25 \times \text{cost\_spent\%}$$

where $\text{coverage\%} = \min\left(100,\ \dfrac{\text{total payload capacity}}{\text{total grid demand}} \times 100\right)$.

Coverage is weighted at 75% because serving demand is the primary goal. Cost is penalised at 25% to discourage unnecessary spending. Any fleet exceeding the budget receives a score of $-\infty$ and is never selected.

0.4.4 Strategy 1 : Brute-Force Search

All valid $(n_{\text{light}}, n_{\text{heavy}})$ combinations within budget are enumerated and scored. With a 15,000-unit budget, approximately 144 combinations are checked. Brute-Force always returns the globally optimal fleet and completes in under one millisecond.

0.4.5 Strategy 2 : Genetic Algorithm

A Genetic Algorithm evolves a population of fleet configurations over 40 generations to find a high-scoring fleet without exhaustive enumeration.

Table 6: Genetic Algorithm components

Component	**Description**
Chromosome	$[n_{\text{light}},\ n_{\text{heavy}}]$, an integer pair representing one fleet
Population	30 random fleet configurations per generation
Selection	Tournament: best of 3 randomly chosen candidates
Crossover	Each gene independently chosen from either parent
Mutation	± 1 on a random gene, clamped to valid range
Elitism	Best individual always preserved into the next generation
Generations	40

0.4.6 Unit Testing

A test suite of 37 unit tests was written and all 37 pass with zero failures. Tests cover every helper function, each CSP rule individually, the fitness function, Brute-Force search, Genetic Algorithm, `compute_total_demand`, and the full `select_fleet` API.

Table 7: Unit test results for Modules 2A and 2B

Test Category	**Tests**	**Result**
Helper functions (manhattan, get_neighbors)	4	4/4 Pass
R1 — Industrial Safety	4	4/4 Pass
R2 — Residential Coverage	5	5/5 Pass
R3 — Hub Charging	4	4/4 Pass
R4 — Medical Access	4	4/4 Pass
validate_layout integration	3	3/3 Pass
Fleet fitness function	3	3/3 Pass
Brute-Force selector	4	4/4 Pass
Genetic Algorithm selector	2	2/2 Pass
compute_total_demand & edge cases	2	2/2 Pass
select_fleet full API	2	2/2 Pass
Total	**37**	**37/37**

0.5 Module 3A — A* Path Planner

0.5.1 Overview

The A* Path Planner (`astar_planner.py`) finds the shortest-cost route between any two cells on the grid. It is used both during initial route planning (before the simulation starts) and during live rerouting when a no-fly zone is activated mid-flight. The planner accounts for zone-based movement restrictions, meaning routes naturally avoid high-cost zones such as schools and hospitals when cheaper alternatives exist.

0.5.2 The A* Algorithm

A* is an informed search algorithm that finds the optimal path by evaluating each candidate cell with:

$$f(n) = g(n) + h(n)$$

where $g(n)$ is the actual accumulated cost from the start cell to cell n (sum of zone weights along the path), and $h(n)$ is the heuristic estimate of the remaining cost from n to the goal. Manhattan distance is used as the heuristic:

$$h(n) = |r_n - r_{\text{goal}}| + |c_n - c_{\text{goal}}|$$

Manhattan distance is *admissible* on a 4-directional grid, meaning it never overestimates

the true remaining cost, which guarantees that A* always returns an optimal path.

0.5.3 How A* Works Step by Step

1. Place the start cell in the open set with priority $f = 0$.
2. Pop the cell with the lowest f value.
3. If this cell is the goal, reconstruct and return the path.
4. Otherwise, expand each 4-directional neighbour: compute the new g cost through the current cell, update if it is cheaper than previously known, and push to the open set with priority $f = g + h$.
5. Repeat until the goal is reached or the open set is empty (no path exists).
6. Path is reconstructed by following `came_from` pointers back from goal to start.

0.5.4 Zone Movement Costs

Rather than treating all cells as equal, drones incur different movement costs based on the zone they fly over. This models real-world airspace restrictions.

Table 8: Zone movement costs used by the A* planner

Zone	Cost	Reason
Open Field	1.0	No restrictions
Residential	1.2	Minor noise constraints
Commercial	1.3	Moderate airspace congestion
Industrial	1.5	Restricted airspace
Hospital	1.8	Strict flight precautions
School	2.0	Highest restriction
No-fly zone	∞	Completely blocked, cannot enter

0.5.5 Public Functions

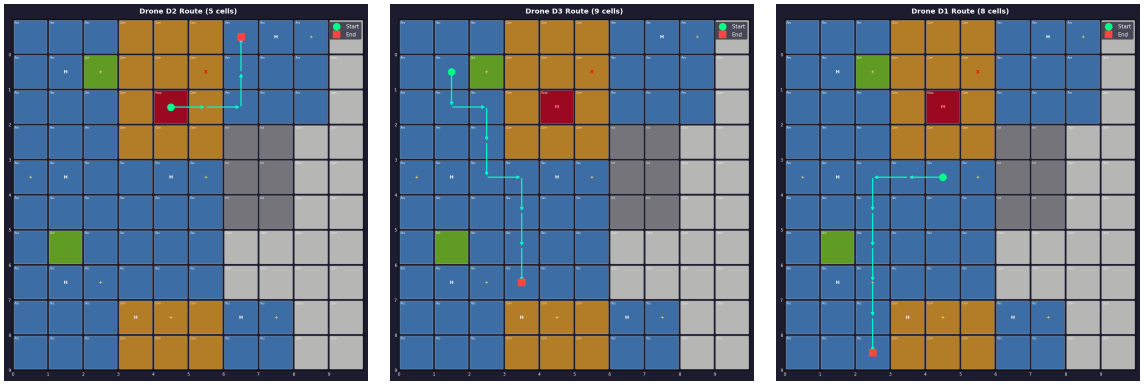
`astar_planner.py` exposes four functions used by other modules:

- `find_path(grid, start, goal)` — Runs A* and returns the optimal route as a list of `(row, col)` positions, or `None` if no path exists.
- `path_cost(grid, route)` — Computes the total weighted cost of a route.

- `find_paths_for_deliveries(grid, deliveries)` — Batch-runs A* for a list of `Delivery` objects and stores each route on the object.
- `reroute_delivery(grid, delivery, current_pos)` — Re-runs A* from the drone's current position when a no-fly zone is activated mid-flight.

0.5.6 Route Visualisations

The three sample deliveries produce the following A* routes. Each figure shows the start cell, destination, and the planned path overlaid on the zone map.



(a) DEL1: (2,4) → (0,6)
5 cells, cost = 4.90

(b) DEL2: (1,1) → (7,3)
9 cells, cost = 9.60

(c) DEL3: (4,4) → (9,2)
8 cells, cost = 8.40

Figure 4: A* route visualisations for the three sample deliveries. Arrows show the planned path; zone colours show movement cost.

0.5.7 Disruption Handling — Rerouting

When a no-fly zone is activated at step 11 of the simulation (cell (1,5) blocked by a weather alert), the planner checks every active drone's remaining route for blocked cells. Any drone whose path passes through the new no-fly zone has `reroute_delivery()` called from its current position. The new route avoids the blocked cell and is adopted immediately.

0.6 Module 3B — Delivery Simulator

0.6.1 Overview

The Delivery Simulator (`delivery_simulator.py`) runs the full 20-step tick-based simulation. Each tick represents one cell of drone movement. The simulator coordinates drone assignment, movement, disruption detection, rerouting, battery monitoring, and anomaly response.

0.6.2 Drone Assignment

Before movement begins, each delivery is matched to a suitable drone using a greedy rule: deliveries are sorted heaviest-first, and each is assigned the first available drone whose payload capacity meets the weight requirement and whose maximum range covers the planned route length. This ensures heavy deliveries are handled by heavy drones while lighter parcels use lighter, cheaper drones.

0.6.3 20-Step Simulation Schedule

Table 9: 20-step simulation event schedule

Steps	Module	What Happens
1	CSP Validator	All 4 constraints checked; layout report printed
2	Fleet Selector	Optimal fleet selected; drones assigned to deliveries
3	A* Planner	Routes computed; lengths and costs logged
4	Simulator	Drone movement begins
5 to 10	Simulator	Each drone advances one cell per tick along its A* route
11	Environment	No-fly zone activated at (1,5), weather alert
12 to 14	Disruption	Blocked paths detected; affected drones rerouted via A*
15 to 17	Monitoring	Battery levels checked; ML anomaly detector called per drone
18 to 19	Simulator	Remaining drones complete their routes
20	Summary	Results compiled; log saved to `results/simulation_log.txt`

0.6.4 Battery and Anomaly Model

Each cell of movement costs **5%** battery. If battery drops below **20%**, the drone is forced to return to its home hub immediately and its delivery is marked failed. With a **5% probability** per tick, a battery anomaly event is triggered, causing $5\times$ faster drain that step.

0.6.5 Event Log

Every event is recorded with step number, module tag, and message. The full log is printed to the terminal and saved to `results/simulation_log.txt`.

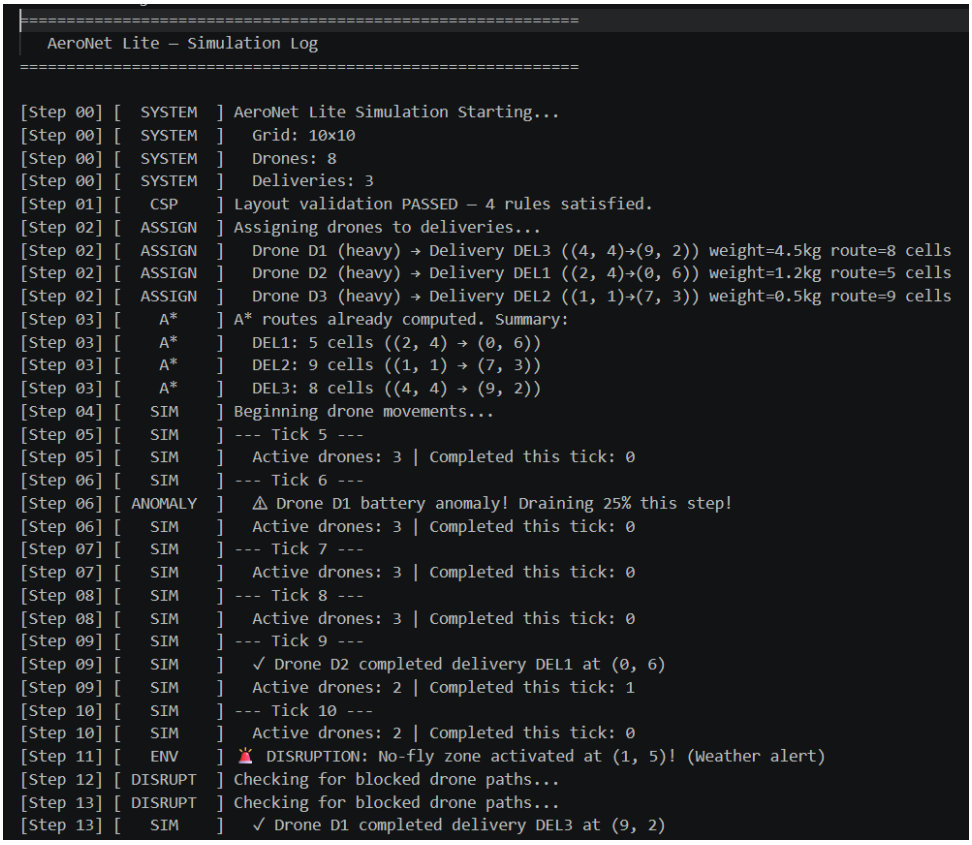


Figure 5: Excerpt from the simulation event log, steps 9 through 20 showing delivery completions, the no-fly disruption at step 11, rerouting at steps 12 to 13, and the final summary.

0.6.6 Simulation Dashboard

The four-panel dashboard figure is generated at the end of each run and saved to `results/figures/dashboard.png`.

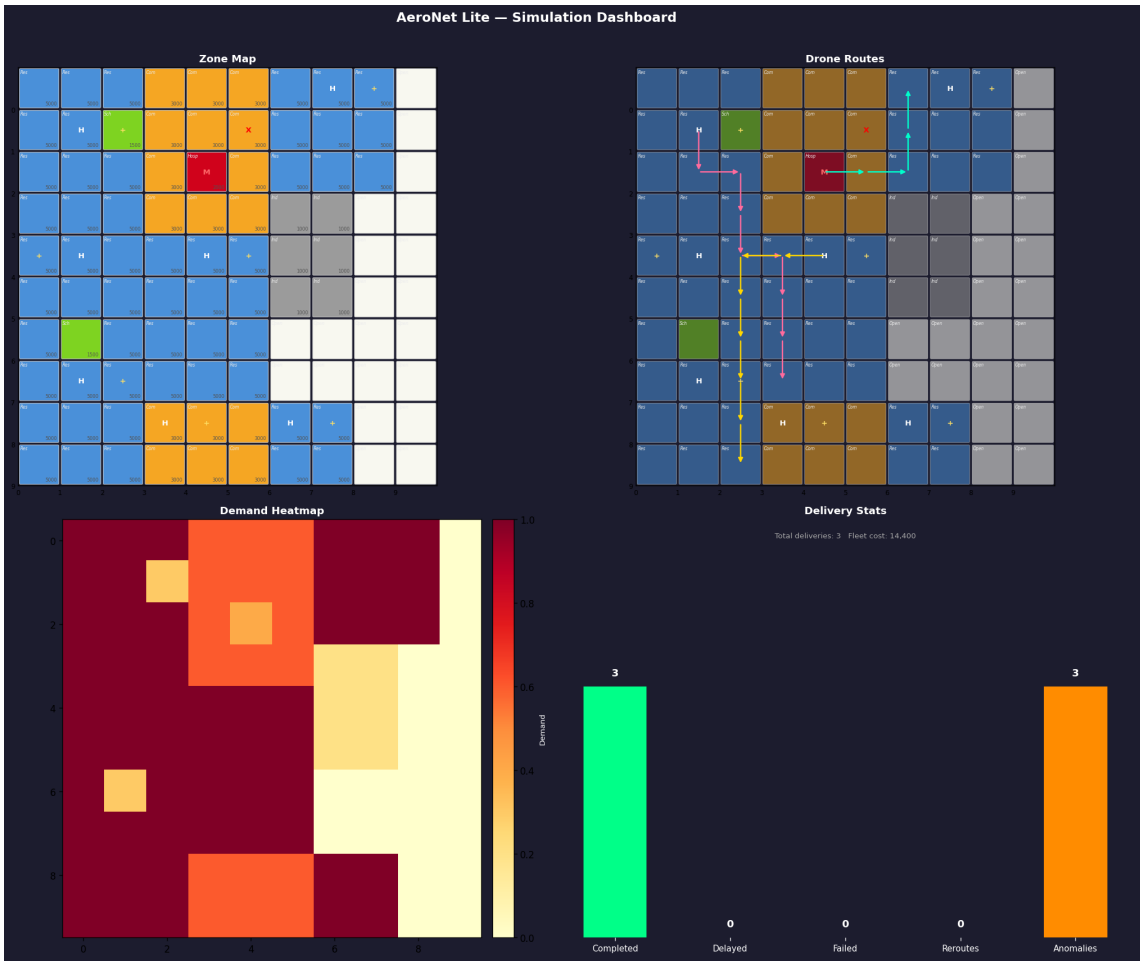


Figure 6: Simulation dashboard, zone map (top-left), all three A* drone routes (top-right), demand heatmap (bottom-left), and delivery statistics bar chart showing 3 completed, 0 failed, 0 delayed (bottom-right).

0.7 Module 4 — ML Pipeline

The ML pipeline (`ml_pipeline.py`) trains two models on first import and exposes three public symbols used by the simulator and dashboard: `get_demand_forecast()`, `detect_anomaly()`, and `DEMAND_BY_AREA`.

0.7.1 Part A — Demand Forecasting

0.7.1.1 Dataset Overview

The Amazon Delivery Dataset contains 3,000 raw records expanded to 43,648 usable rows after cleaning. It covers agent attributes (age, rating), environmental conditions (weather, traffic), delivery area, vehicle type, order timestamps, GPS coordinates for store and drop-off locations, and the target variable, delivery time in minutes.

0.7.1.2 Data Cleaning

Several issues were resolved before modelling:

- Missing `Weather` values filled with mode ("Cloudy").
- Missing `Agent_Rating` values replaced with column median (4.7).
- Rows where `Traffic` contained the string literal "NaN" were dropped.
- `Order_Date` and `Order_Time` parsed to extract `Day_of_Week`, `Month`, and `Order_Hour`.
- `Distance_km` engineered from GPS coordinates using the Haversine formula.
- Categorical columns (Weather, Traffic, Area, Vehicle) encoded with Label Encoding.

Final dataset: 43,648 rows, 10 features. 80% training / 20% test split.

0.7.1.3 Exploratory Data Analysis

Six EDA plots were produced to understand the data before modelling: delivery time distribution, traffic boxplot, weather bar chart, area breakdown, distance vs. delivery time scatter, and hourly demand pattern. Key findings: jam traffic conditions produce the longest delays; Metropolitan zones carry the highest average demand; distance alone is a weak predictor compared to traffic and area.

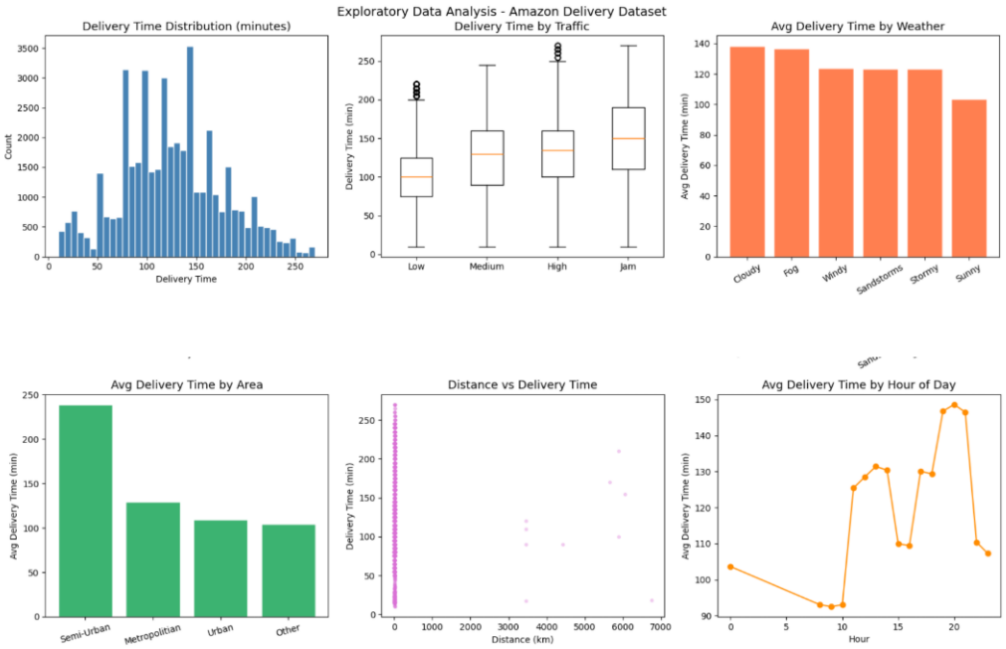


Figure 7: EDA outputs, delivery time distribution, traffic impact, weather effect, area comparison, distance scatter, and hourly demand pattern.

0.7.1.4 Model Training and Comparison

Two regression models were trained and compared:

Table 10: Demand forecasting model comparison

Model	MAE (min)	RMSE (min)	Verdict
Linear Regression	34.82	45.17	Baseline, too coarse
Random Forest	25.56	36.45	Selected, more accurate

Random Forest reduces MAE by 26% over Linear Regression. Feature importance analysis shows `Distance_km` (23.2%), `Agent_Rating` (20.0%), and `Weather_enc` (15.9%) are the top three predictors.

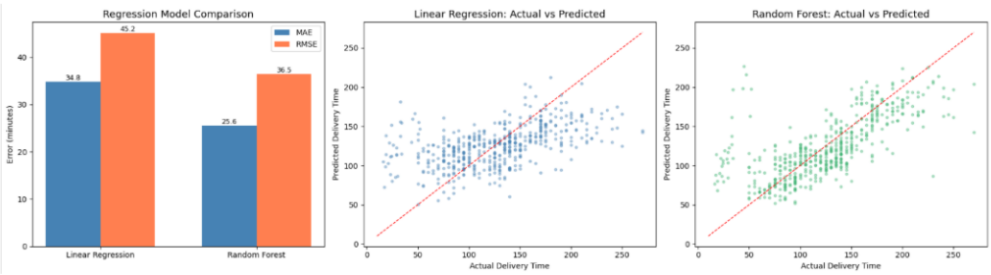


Figure 8: Model comparison, MAE/RMSE bar chart, Linear Regression actual vs. predicted, and Random Forest actual vs. predicted.

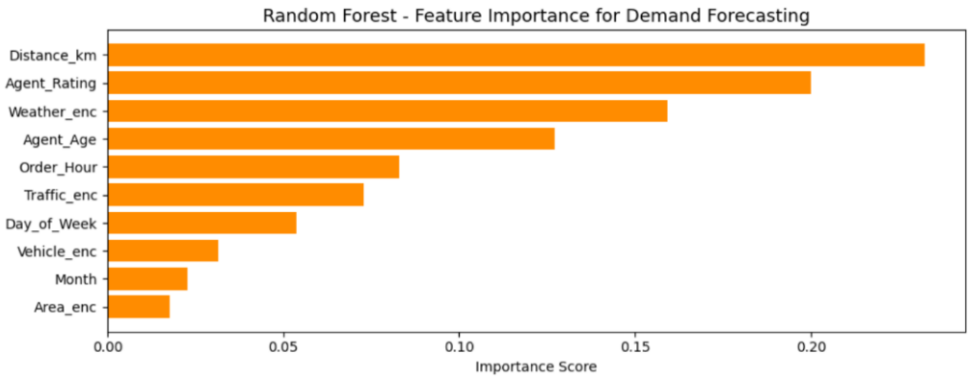


Figure 9: Random Forest feature importances. Distance and agent rating dominate; time-of-day features contribute minimally.

0.7.1.5 Demand Mapping and Simulation Integration

Average delivery time per area type is extracted and stored as `DEMAND_BY_AREA`:

Table 11: DEMAND_BY_AREA values used by the grid model

Area Type	Avg Delivery Time (min)	Demand Level
Metropolitan	129.7	Highest
Urban	104.5	High
Other	126.7	Medium
Semi-Urban	238.6	Variable

`get_demand_forecast()` is called at simulation steps 15 and 16 to dynamically adjust fleet deployment. A stormy, jam-traffic Metropolitan delivery at 5pm returns 197.2 minutes (highest demand); a sunny, low-traffic Urban delivery at 8am returns 106.9 minutes.

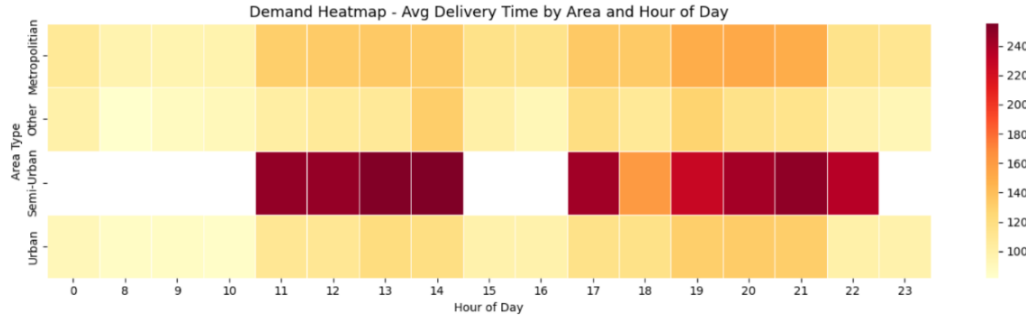


Figure 10: Average delivery time by area type and hour of day, darker cells indicate higher demand periods. Metropolitan zones show the heaviest load throughout the day.

0.7.2 Part B — Anomaly Detection

0.7.2.1 Synthetic Telemetry Dataset

Real labelled UAV telemetry was not available, so a synthetic dataset of 2,000 samples (500 per class) was generated with statistically distinct distributions for each anomaly type:

Table 12: Anomaly classes and key telemetry signals

Class	Key Signal	Samples
Normal	All features within typical operating ranges	500
Battery_Anomaly	`battery_drop` unusually high (mean 12%)	500
Route_Anomaly	`route_deviation` unusually high (mean 8 cells)	500
Sensor_Spike	Large `altitude_change` and `speed_change` simultaneously	500

0.7.2.2 Feature Distributions

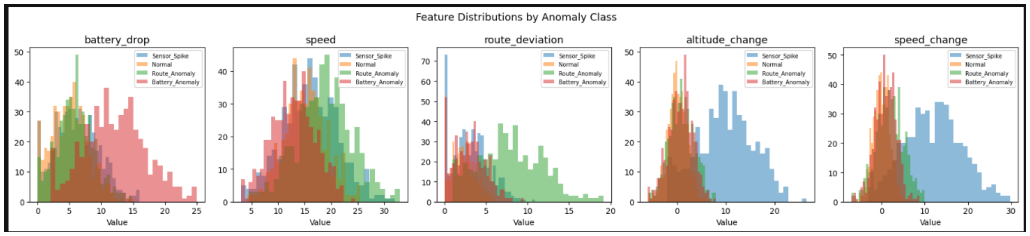


Figure 11: Feature distributions by anomaly class, each feature reveals clear separation for the class it is designed to detect. `battery_drop` separates Battery_Anomaly; `route_deviation` separates Route_Anomaly; `altitude_change` and `speed_change` jointly separate Sensor_Spike.

0.7.2.3 Model Training and Comparison

Four classifiers were trained on the same 80/20 stratified split:

Table 13: Anomaly classifier comparison

Classifier	Test Accuracy	Notes
Decision Tree	79.75%	Fast; prone to overfitting
Random Forest	82.75%	Selected, balanced per-class performance
KNN	79.50%	Lowest; sensitive to scale
Naive Bayes	82.75%	Tied with RF; assumes feature independence

Random Forest was selected over Naive Bayes despite equal overall accuracy because its per-class F1 scores are more balanced across all four anomaly types.

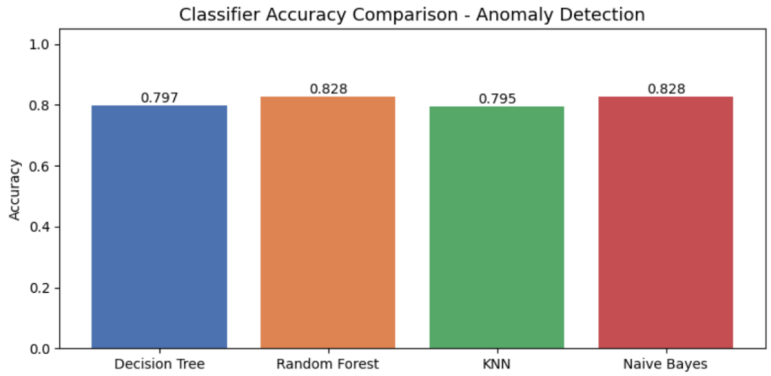


Figure 12: Classifier accuracy comparison, Random Forest and Naive Bayes share the top position at 82.75%.

0.7.2.4 Detailed Evaluation — Random Forest

Table 14: Random Forest classification report on the test set (400 samples)

Class	Precision	Recall	F1-score	Support
Battery_Anomaly	0.79	0.78	0.78	100
Normal	0.80	0.78	0.79	100
Route_Anomaly	0.80	0.80	0.80	100
Sensor_Spike	0.89	0.92	0.91	100
Weighted avg	0.82	0.83	0.82	400

Sensor_Spike is the easiest class to detect (F1 = 0.91) because its altitude and speed signals are the most distinct. Battery_Anomaly and Normal are harder to separate (F1 = 0.78 to 0.79) due to overlapping battery_drop distributions in the mid-range.

0.7.2.5 Confusion Matrices

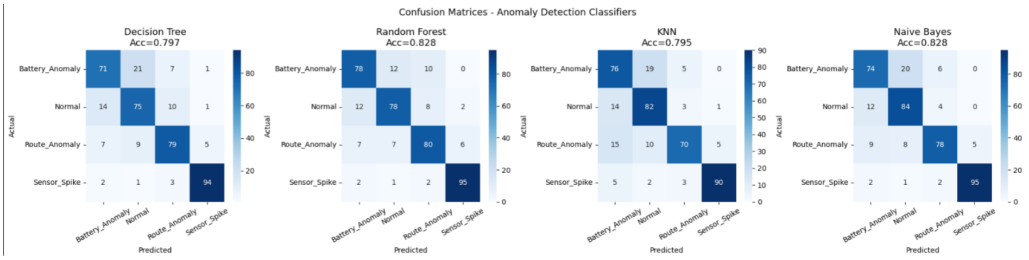


Figure 13: Confusion matrices for all four classifiers. Diagonal cells are correct predictions; off-diagonal are misclassifications. All classifiers struggle most with Battery_Anomaly vs Normal.

0.7.2.6 Cross-Validation

5-fold cross-validation on the full 2,000-sample dataset confirmed consistent generalisation. The mean accuracy of 0.831 is close to the single test accuracy of 0.828, and the standard deviation of 0.015 is very low, confirming the model does not overfit.

Table 15: Random Forest 5-fold cross-validation results

Metric	Value
Per-fold accuracies	[0.808, 0.850, 0.830, 0.845, 0.823]
Mean accuracy	0.831
Std deviation	0.015

0.7.2.7 Simulation Integration

`detect_anomaly()` is called at simulation steps 15 to 19. The Disruption Handler responds to each anomaly class as follows:

Table 16: Anomaly detection responses in the simulation

Anomaly Class	Simulation Response
`Battery_Anomaly`	Force drone to return to nearest hub immediately
`Route_Anomaly`	Recalculate path via A* from current position
`Sensor_Spike`	Ground drone pending sensor diagnostics
`Normal`	Continue on planned route

0.8 Module 6 — Streamlit Dashboard

0.8.1 Overview

`app.py` provides an interactive web dashboard that ties all five modules together. It is launched from the project root and opens at `http://localhost:8501`.

0.8.2 Dashboard Tabs

The interface is organised into six tabs:

Table 17: Streamlit dashboard tab structure

Tab	Content
1 — Grid & Validation	Load the 10×10 grid, run CSP validation, and view the zone map with pass/fail results per rule.
2 — Fleet Selection	Adjust budget slider and choose Brute-Force or GA. Displays coverage, cost, and fitness score.
3 — Route Planning	Enter custom pickup and dropoff coordinates. Runs A* and overlays routes on the grid.
4 — Simulation	Runs the full 20-step simulation. Shows completion, failure, and delay counts.
5 — Results & Logs	Full simulation event log, all saved figures, and download buttons.
6 — ML Pipeline	Demand forecast with sliders for hour, weather, traffic, area, and distance. Anomaly detector with sliders for all five telemetry readings.

0.8.3 Quick Run

A “Run Full Simulation” button in the sidebar executes the complete pipeline, including grid loading, CSP validation, fleet selection, A* routing, and the 20-step simulation, in a single click with live progress spinners.

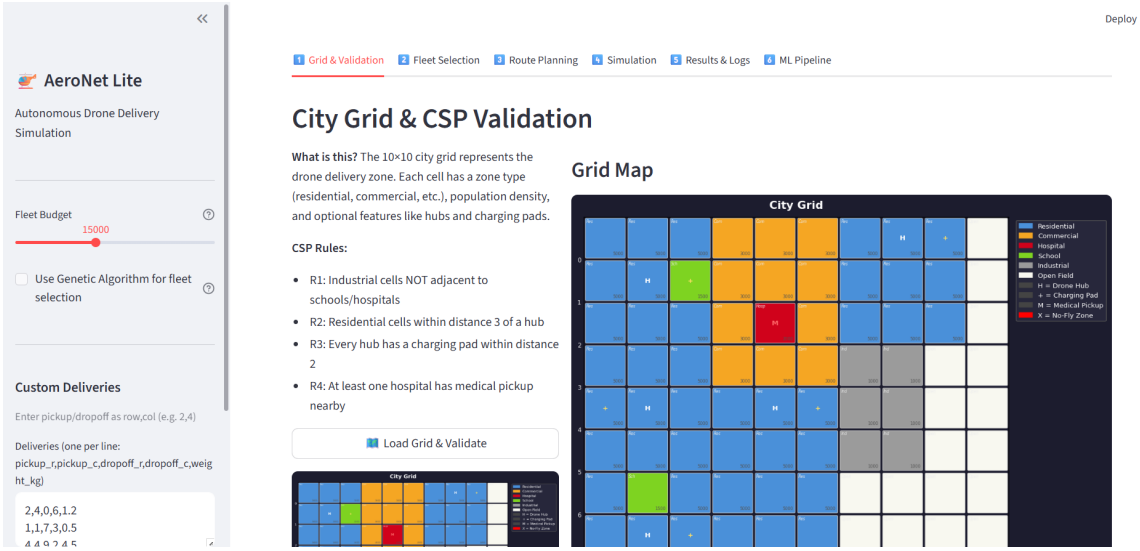


Figure 14: Dashboard Tab 1, Grid and CSP Validation. The zone map and validation report are shown side by side.

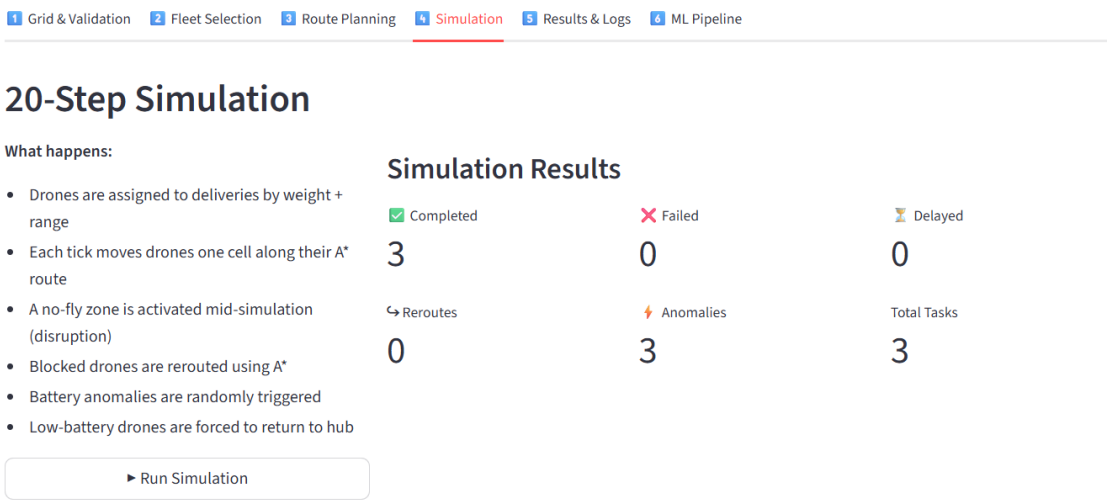
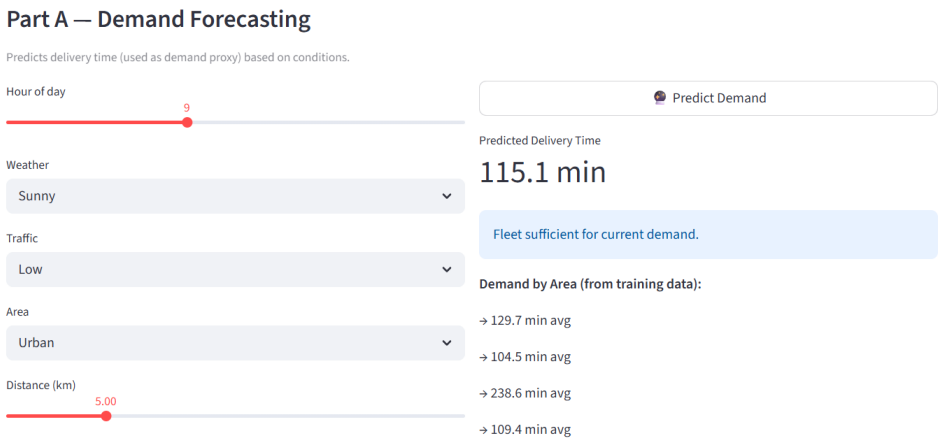
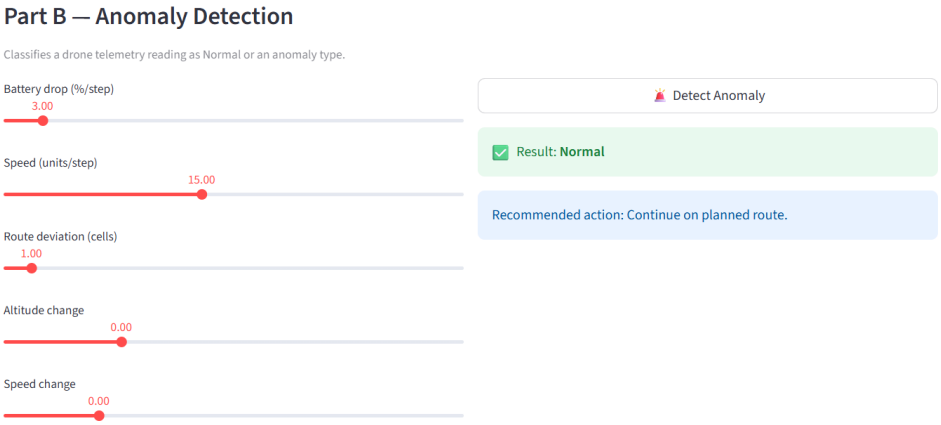


Figure 15: Dashboard Tab 4, Simulation results showing 3 completed deliveries, 0 failed, 0 delayed.



(a) Upper Panel: Demand forecast inputs and prediction.



(b) Lower Panel: Anomaly detection sliders and classification result.

Figure 16: Dashboard Tab 6, ML Pipeline interface.

0.9 Results

0.9.1 Simulation Outcomes

The baseline 20-step simulation with three deliveries and a 15,000-unit budget consistently produces the following results:

Table 18: Baseline simulation results

Metric	Value
Total deliveries	3
Completed	3
Failed	0
Delayed	0
Reroutes	1 (step 12, no-fly disruption)
Fleet cost	14,400 units
Grid demand coverage	61.16%

0.9.2 Output Files

Table 19: Generated output files

File	Contents
`results/validation_report.txt`	CSP rule check results with violations and fixes
`results/fleet_selection_result.txt`	Fleet metrics: drones, cost, coverage, fitness
`results/simulation_log.txt`	Full 20-step event log
`results/figures/grid_map.png`	City grid zone map
`results/figures/demand_heatmap.png`	Demand heatmap
`results/figures/route_D1/D2/D3.png`	Individual A* route overlays
`results/figures/dashboard.png`	Full four-panel simulation dashboard

0.10 How to Run

1. Install all dependencies (run once):

   ```
   pip install -r requirements.txt
   ```

2. Run the full CLI simulation (generates all result files):

   ```
   python src/main.py
   ```

3. Launch the interactive Streamlit dashboard:

```
python -m streamlit run app.py
```

Then open `http://localhost:8501` in your browser.

The CSV file `data/raw/amazon_delivery.csv` is optional. Without it, the demand model uses fixed fallback values and the anomaly model trains on synthetic data as normal, and all other modules work identically.

0.11 Conclusion

AeroNet Lite successfully demonstrates how multiple AI techniques combine to address a realistic urban drone delivery problem. The CSP validator enforces hard spatial safety rules that mirror real airspace regulations. The Genetic Algorithm fleet selector shows how evolutionary search finds near-optimal solutions without exhaustive enumeration. A* with zone-weighted movement costs produces routes that respect real-world restrictions rather than blindly minimising distance. The 20-step simulation handles dynamic disruptions through live rerouting and battery management. The Random Forest models bring data-driven decision making into the loop, where demand forecasting informs fleet deployment and anomaly detection enables automatic safety responses. The Streamlit dashboard brings everything together into a usable interface.

The modular architecture means each component can be extended or replaced independently. Future directions include integrating real UAV telemetry, expanding to larger city grids, adding multi-drone collision avoidance, and connecting to real-time weather APIs for dynamic no-fly zone updates.

Bibliography

[1] Russell, S. and Norvig, P. (2020). *Artificial Intelligence: A Modern Approach*, 4th ed. Pearson.

[2] Hart, P. E., Nilsson, N. J., and Raphael, B. (1968). A formal basis for the heuristic determination of minimum cost paths. *IEEE Transactions on Systems Science and Cybernetics*, 4(2), 100–107.

[3] Breiman, L. (2001). Random forests. *Machine Learning*, 45(1), 5–32.

[4] Holland, J. H. (1975). *Adaptation in Natural and Artificial Systems*. University of Michigan Press.

[5] Kumar, V. (1992). Algorithms for constraint-satisfaction problems: A survey. *AI Magazine*, 13(1), 32–44.

[6] Streamlit Inc. (2024). *Streamlit Documentation*. https://docs.streamlit.io

[7] Scikit-learn Developers (2024). *scikit-learn: Machine Learning in Python*. https://scikit-learn.org

[8] Amazon Delivery Dataset. Kaggle. https://www.kaggle.com/datasets